2.6. Implementación

La solución se organiza en una plataforma web de gestión clínica, una API de servicios, una base de datos relacional y un componente móvil orientado a la consulta del odontólogo. Esta separación permite concentrar las operaciones administrativas en la web y delimitar la experiencia móvil a los pacientes asignados y sus estudios radiográficos. CERPAX participa como proveedor externo mediante un portal de entrega con enlace temporal.

2.6.1. Plataforma web y gestión clínica

La interfaz web se implementó con Angular y TypeScript. Los componentes de Angular Material se utilizaron para construir formularios, tablas, diálogos de confirmación y elementos de navegación. La aplicación consume los servicios de la API mediante solicitudes HTTP y presenta las acciones disponibles según los permisos de la sesión.

Se implementaron operaciones de registro, consulta y actualización de pacientes y odontólogos, así como la gestión de horarios y citas. En las citas se valida que el intervalo solicitado corresponda al horario del profesional y se rechazan las superposiciones. Las consultas incorporan paginación y búsqueda para evitar la transferencia de todos los registros en una sola respuesta.

Los pacientes y los odontólogos comparten la tabla `usuarios`, pero su función se distingue mediante las asociaciones del sistema. La tabla `pacientes_odontologos` relaciona al paciente con el profesional y la clínica correspondiente. Esta asignación se utiliza para delimitar los registros que puede consultar cada odontólogo. El paciente se representa como un registro clínico y no como un actor con acceso al panel.

2.6.2. Autenticación y control de acceso

El backend se desarrolló con Node.js y Express. El inicio de sesión de la plataforma web se realiza mediante `/api/login`; la contraseña se verifica con bcrypt y, si las credenciales son válidas, se emite un token JWT con vigencia de ocho horas. Las solicitudes protegidas incluyen el token en el encabezado de autorización.

El middleware de autenticación comprueba la firma, la vigencia del token, el estado del usuario y la versión de su sesión. Posteriormente se verifica el permiso requerido y la pertenencia a la clínica seleccionada. De esta manera, ocultar una opción en la interfaz no constituye el único control: las restricciones también se aplican en la API y en las consultas de datos.

El administrador dispone de funciones de administración de cuentas y permisos. El odontólogo consulta la información correspondiente a sus pacientes asignados. El acceso de recepción se limita a datos administrativos, citas y asignaciones, excluyendo los campos clínicos y las operaciones de administración de cuentas.

El siguiente fragmento muestra la comprobación de permisos que utiliza el backend. La función `can` comprueba el permiso de la operación o el rol de administrador; el middleware rechaza la solicitud con el código HTTP 403 cuando la comprobación falla. La excepción `bypass` corresponde al acceso local de desarrollo, condicionado por la configuración del servidor, y no habilita el acceso local en producción.

Fragmento 1. Control de permisos por operación.

```
export const can = (user, key, action) => !!user && (
  user.bypass || user.roles.includes('Super Admin') ||
  user.permissions.includes(`${key}.${action}`)
);
export function requirePermission(key, action) {
  return (req, res, next) => {
    try {
      if (!can(req.user, key, action)) {
        fail('No tienes permiso para realizar esta acción', 403);
      }
      next();
    } catch (e) { next(e); }
```

```
  };
}
```

Fuente: código del proyecto, `backend/src/auth.js`; formato ajustado para su lectura.

2.6.3. Solicitudes y entrega de radiografías

El odontólogo registra una solicitud radiográfica indicando el paciente, el tipo de estudio y las indicaciones. La API comprueba la asignación antes de crear el registro. Las solicitudes utilizan los estados *Pendiente*, *Completada* y *Cancelada*.

Para la entrega de CERPAX se implementó la generación de un enlace y un código QR con vigencia configurable entre una y setenta y dos horas. El token se genera mediante bytes aleatorios y la base de datos conserva su hash, junto con las fechas de vencimiento, revocación y utilización. Al generar un nuevo acceso se revocan los enlaces anteriores que todavía no han sido utilizados.

El proveedor externo no necesita iniciar sesión en el panel. El portal valida el token y solicita una URL firmada para cargar el archivo en Supabase Storage. Una vez transferido el archivo, la API verifica sus metadatos y la correspondencia con el acceso temporal. La confirmación registra la radiografía, completa la solicitud y marca el acceso como utilizado dentro de una transacción.

Se admiten archivos PDF, PNG, JPEG y WebP con un tamaño máximo de 50 MiB. Los objetos se almacenan en un bucket privado y sus metadatos se conservan en PostgreSQL. La visualización o descarga desde el panel requiere autorización; conocer el identificador de una solicitud no habilita por sí mismo el acceso al archivo.

El segundo fragmento muestra la validación del acceso temporal. La consulta compara el hash del token y exige que el enlace no haya vencido, sido revocado o utilizado, y que la solicitud permanezca pendiente. Si no se encuentra un acceso válido, la API devuelve el código HTTP 410. El parámetro `lock` permite bloquear las filas durante las operaciones que modifican el estado de la solicitud dentro de una transacción.

Fragmento 2. Validación del enlace temporal de CERPAX.

```
async function access(client, token, lock = false) {
  const {rows} = await client.query(
    `SELECT a.id AS acceso_id,a.vence_en,a.storage_path,s.*
     FROM accesos_radiografias a JOIN solicitudes_radiografias s
       ON s.id=a.solicitud_id
     WHERE a.token_hash=$1
       AND a.vence_en>now()
       AND a.revocado_en IS NULL
       AND a.usado_en IS NULL
       AND s.estado='Pendiente'
     ${lock ? 'FOR UPDATE OF s,a' : ''}`,
    [tokenHash(token)]
  );
  if (!rows[0]) fail(
    'El enlace venció, fue revocado o ya fue utilizado',
    410
  );
  return rows[0];
}
```

Fuente: código del proyecto, `backend/src/radiography.js`; formato ajustado para su lectura.

2.6.4. Componente móvil para el odontólogo

La aplicación móvil DentalCare utiliza Flutter y Dart para la interfaz Android. Dentro de la solución, su alcance se centra en el inicio y cierre de sesión del odontólogo, la consulta de pacientes asignados, la consulta del estado de las solicitudes y la visualización de las radiografías disponibles. Esta delimitación permite utilizar el teléfono como medio de consulta clínica sin trasladar todas las operaciones del panel web.

El diseño de integración establece la API de la plataforma como punto de acceso a los datos y como responsable de verificar la identidad, los permisos y las asignaciones. La administración de cuentas, roles y permisos, así como la gestión de odontólogos, horarios y citas, se concentra en la plataforma web. El paciente no inicia sesión en la aplicación móvil y CERPAX mantiene el acceso mediante el enlace temporal de entrega.

2.6.5. Decisiones técnicas y resolución de dificultades

Una dificultad técnica del flujo radiográfico consiste en mantener la coherencia entre el archivo almacenado y la solicitud clínica. Para atenderla, se separaron la preparación de la carga, la transferencia del archivo y la confirmación en la API. La solicitud se completa después de verificar el objeto cargado; los cambios relacionados en PostgreSQL se ejecutan en una transacción. La restricción de unicidad de `solicitud_id` en `radiografias` impide registrar más de una radiografía para la misma solicitud.

Otra dificultad consiste en restringir el acceso cuando distintos profesionales utilizan el mismo sistema. Se resolvió incorporando filtros por clínica y por asignación de pacientes en las consultas del backend, además de las comprobaciones de permisos por operación. Las solicitudes fuera del alcance autorizado se rechazan o se excluyen de los resultados.

Para delimitar el alcance funcional se retiraron del catálogo y de la navegación los módulos de contabilidad, laboratorio y departamentos, junto con los roles de contador y laboratorista. Las tablas heredadas se conservaron para evitar la pérdida de información y mantener la compatibilidad con registros existentes. Esta decisión distingue la retirada de una función visible de la eliminación física de sus datos.

Los cambios del esquema se gestionan mediante migraciones SQL versionadas. Las consultas emplean parámetros para los valores recibidos y los nombres de recursos se resuelven desde el catálogo permitido. Asimismo, los errores de validación se presentan mediante respuestas estructuradas, mientras que los errores internos evitan revelar credenciales o detalles de conexión.

2.6.6. Verificación de la implementación

La verificación del backend incluyó pruebas de autenticación, validación de datos, gestión clínica, migraciones, restricciones de recepción y flujo radiográfico. Se comprobaron, entre otros escenarios, el acceso de un odontólogo a pacientes no asignados, los enlaces vencidos o revocados, la carga única y las restricciones entre clínicas. En la ejecución local documentada se aprobaron veinticinco pruebas y se completó la compilación de producción de Angular. Estos resultados corresponden al backend y a la aplicación web verificados, y no constituyen una medición de rendimiento ni una prueba de ejecución de la aplicación móvil.

El código completo, las migraciones y las pruebas se mantienen en el repositorio del proyecto. Los diagramas de arquitectura, datos y casos de uso complementan la descripción de los componentes y sus relaciones.